

Finite State Machines (Part 2)

Lecture 05

Josh Brake

Harvey Mudd College

Outline

- Synchronizers, metastability, and the MTBF equation
- Three kinds of state machine: canonical, non-canonical, communicating
- Worked example: a square wave with an adjustable duty cycle
- Build 1: switch debouncer (together)
- Build 2: keypad scanner, Lab 3 prep (in groups)
- A machine that breaks the pattern

Learning Objectives

By the end of this lecture you should be able to...

- Explain what metastability is, and use the MTBF equation to justify how many synchronizer flops a design needs.
- Tell the three structural flavors of state machine apart, and say why you would write one rather than another.
- Decompose a timing problem into a small control FSM plus a counter, and design the signals between them.
- Run the six-step FSM design process on an unfamiliar problem without being walked through it.
- Write a Verilog testbench for a design whose inputs depend on its own outputs.

Synchronizers and Metastability

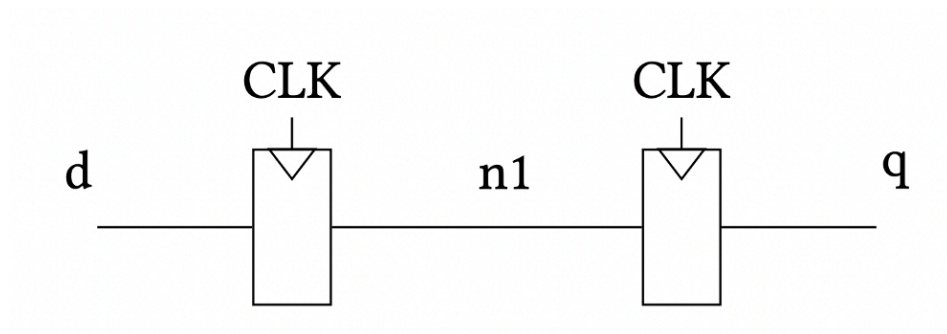
The World Does Not Care About Your Clock

Buttons, keypads, sensors, and other boards change whenever they like. Sooner or later one of them changes **inside the setup/hold aperture** of a flop.

When that happens the flop does not output a 0 or a 1. It sits near the switching threshold in a **metastable** state and stays there for an unbounded time.

“Unbounded” is the important word. There is no maximum resolution time you can look up in the datasheet, because there is no maximum. All you get is a probability that decays with time.

The Two-Flop Synchronizer



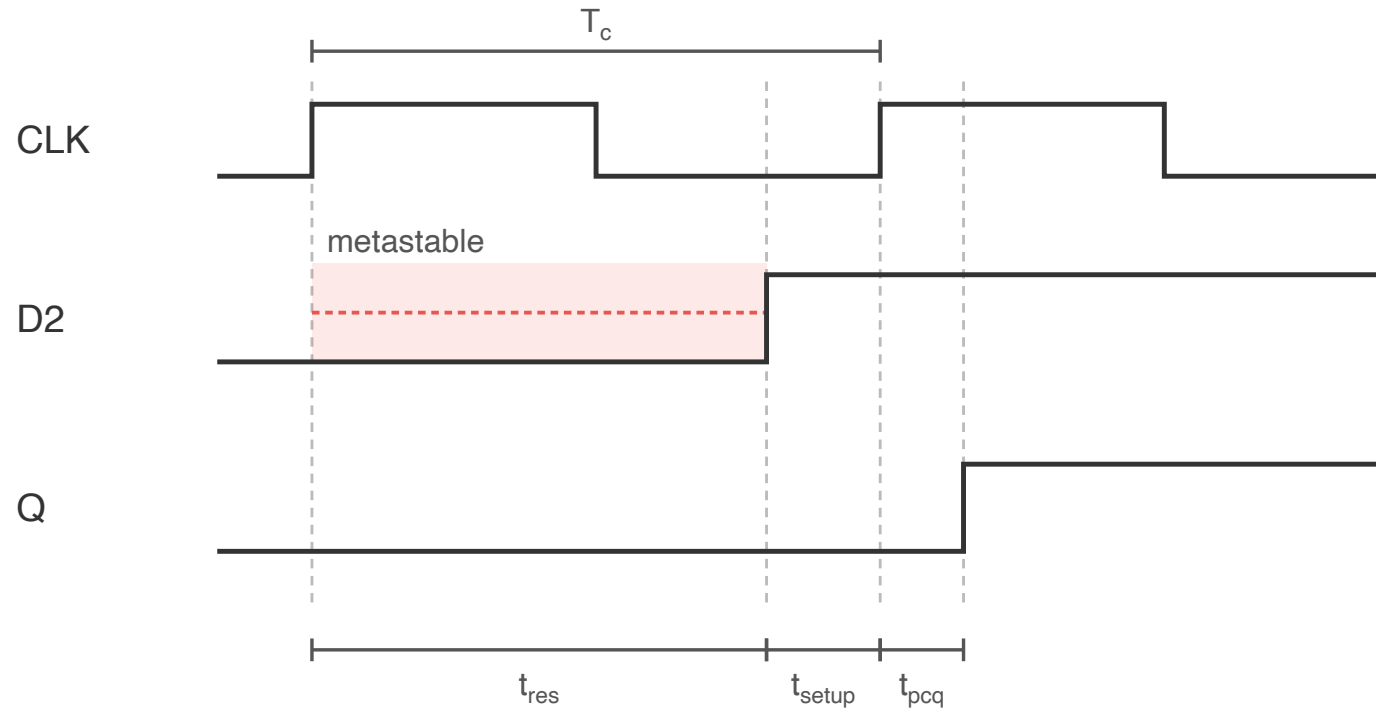
```
1 always_ff @(posedge clk) begin
2     n1 <= d;      // Harris calls this
3     q  <= n1;
4 end
```

Two flops, **F1** and **F2**, with the internal node **D2** between them.

The trick is not that **F1** stops going metastable — it still does. The trick is that **D2** gets a **full clock period** to settle before **F2** looks at it.

Q is then clean, at the cost of **one cycle of latency**.

When Does a Synchronizer Fail?



Synchronizer fails if $t_{res} > T_c - t_{setup}$

The synchronizer **fails** if **Q** itself comes out metastable. That happens when **D2** has not resolved by the time it has to set up at **F2**:

$$t_{res} > T_c - t_{setup}$$

From Probability to MTBF

Start from how long metastability lasts: after waiting t , the flop is still metastable with probability $\frac{T_0}{T_c} e^{-t/\tau}$.

Substitute the time we actually wait, $T_c - t_{setup}$, to get the failure probability **per input change**:

$$P(\text{failure}) = \frac{T_0}{T_c} e^{-\frac{T_c - t_{setup}}{\tau}}$$

If D changes N times a second, failures per second are N times as likely:

$$P(\text{failure})/\text{sec} = N \frac{T_0}{T_c} e^{-\frac{T_c - t_{setup}}{\tau}}$$

MTBF is just the reciprocal of that:

$$\text{MTBF} = \frac{1}{P(\text{failure})/\text{sec}} = \frac{T_c e^{\frac{T_c - t_{setup}}{\tau}}}{NT_0}$$

Reading the Equation

Term	Meaning
T_c	clock period — how long we let D2 resolve
t_{setup}	setup time of F2, which eats into that budget
τ	resolution time constant; how fast metastability decays
T_0	aperture width, so T_0/T_c is the chance a randomly timed edge causes trouble at all
N	times per second that D changes

Read the shape of it: MTBF is **exponential** in the time you wait and only **linear** in everything else. That single fact decides every design choice you make here — which is why the fix is always “wait one more clock cycle,” never “use a better flop.”

Putting Numbers In It

Take $\tau = 0.1$ ns, $T_0 = 0.01$ ns, $t_{setup} = 0.5$ ns, and an input that changes $N = 10$ times a second. Then vary only the clock period:

T_c	f_{clk}	$T_c - t_{setup}$	MTBF
2 ns	500 MHz	1.5 ns	2 years
5 ns	200 MHz	4.5 ns	6×10^{13} years
10 ns	100 MHz	9.5 ns	6×10^{35} years
20.8 ns	48 MHz	20.3 ns	1×10^{83} years

Two flops at 500 MHz fails about **every two years**. That is a real bug in a real product. Our 48 MHz design is safe by a margin that stopped meaning anything a long time ago.

The Fix Is Always One More Cycle

At 500 MHz, add a third flop. Now D2 gets $2T_c - t_{setup} = 3.5$ ns instead of 1.5 ns, and the same equation gives:

$$\text{MTBF} = \frac{T_c e^{(2T_c - t_{setup})/\tau}}{NT_0}$$

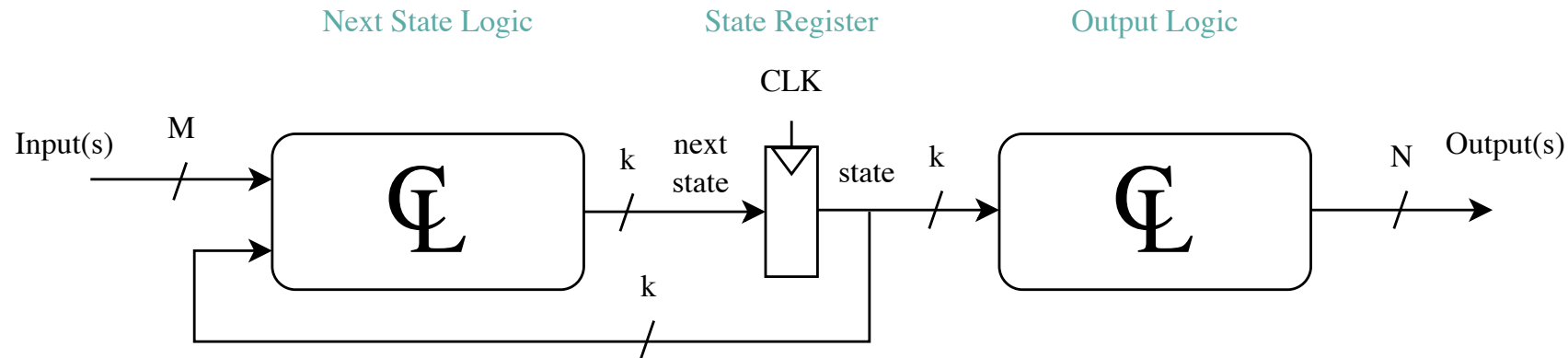
which works out to about 10^9 years.

Two nanoseconds of extra patience bought a factor of $e^{20} \approx 5 \times 10^8$.

Nothing about the flip-flop changed — τ and T_0 are properties of the process, and you cannot buy your way out of them. The only knob you control is how long you are willing to wait, and it is an exponential knob. This is why the answer to “is my synchronizer deep enough?” is always a frequency question, never a parts question.

Three Kinds of State Machine

Type 1: The Canonical FSM



Three blocks, always in this order, always written as three separate chunks of Verilog:

1. **Next state logic** — `always_comb` with a `case` on the current state
2. **State register** — `always_ff` with a reset
3. **Output logic** — `always_comb` or `assign`

- Moore: output depends on state alone.
- Mealy: output depends on state *and* input, so it can change between clock edges.

Both fit this diagram; Mealy just adds an arrow from the input into the output logic block.

Type 1: When to Reach for It

Use the canonical form when the state space is **small and irregular** — when the interesting content of the design is *which state goes where*, and you want that visible.

What you get: the state diagram and the Verilog are the same object. Anyone can check one against the other.

What it costs: one **case** arm per state. Fine for 3 states, unusable for 300.

Type 2: The Non-Canonical FSM

A counter is a state machine. It has state, it has next state logic, and it has output logic.

```
1 always_ff @(posedge clk, posedge reset)
2     if (reset) count <= 0;
3     else count <= count + 1;
```

But we don't write it as a `case` statement, because a 20-bit counter has 1,048,576 states.

Look at what that one line contains: `count` is the state register, `count + 1` is the next state logic, and `count` itself is the output. All three blocks are still there — they have just been fused into an expression, because the next-state function is *regular* enough to write as arithmetic.

Type 2: Same Hardware, Different Description

This is not a different kind of circuit. Both types synthesize to the same thing: flip-flops with combinational logic in front of them.

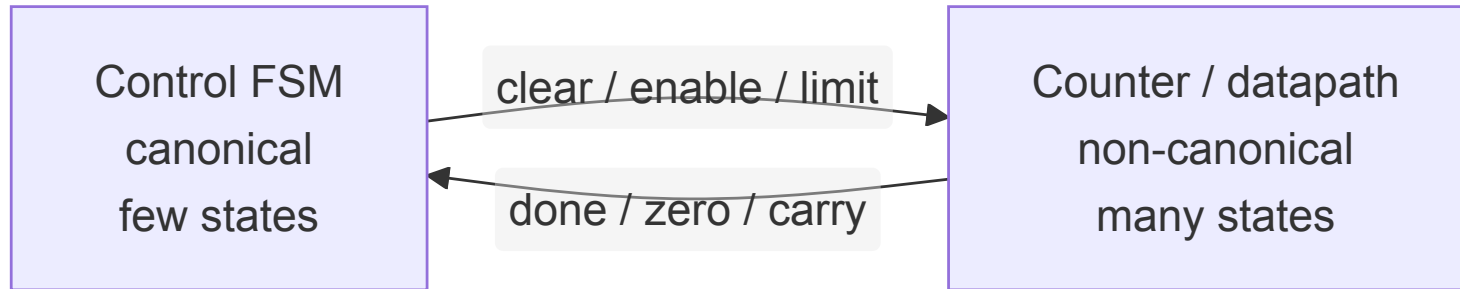
What changes is whether you *enumerate* the transitions or *compute* them.

Non-canonical machines you already have:

- Clock divider (Lab 1–3)
- The rotating **rows** register in the keypad scanner
- Shift registers, LFSRs
- The counter we are about to build

Rule of thumb: if you can describe the next state with an equation, do. If you have to draw it, draw it.

Type 3: Communicating FSMs



Rather than build one machine, build two and let them talk.

- **Down:** control signals — `clear`, `enable`, `load`, a limit value
- **Up:** status signals — `done`, `zero`, `carry`

Type 3: Why Bother Splitting?

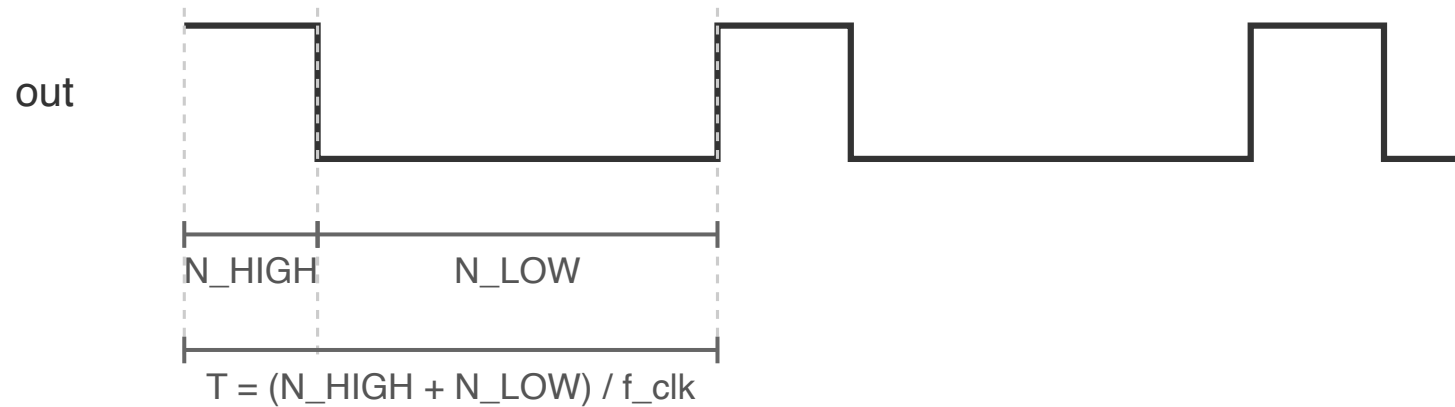
The machine we are about to build has a 2-state controller and a counter that reaches 36,000. Fuse them and you have **72,000 states**.

Kept separate, it is a 2-bubble diagram plus one line of arithmetic.

This is the single most useful idea in the lecture. Almost every real digital system is a small, irregular controller steering one or more large, regular datapath elements. Once you can see that split, most designs stop looking intimidating.

Type 3 in Practice: Variable Duty Cycle

The Specification



Generate a square wave at frequency f_{out} with duty cycle D , from a 48 MHz clock.

$$f_{out} = \frac{f_{clk}}{N_H + N_L} \quad D = \frac{N_H}{N_H + N_L}$$

The catch: the two states have to last **different amounts of time**.

Step 1: Two Machines, Not One

The controller (canonical, Type 1)

Two states, **HIGH** and **LOW**. It owns the output and decides when to flip.

State	out
HIGH	1
LOW	0

The counter (non-canonical, Type 2)

Counts clock cycles. Knows nothing about duty cycle, frequency, or what the output is doing.

Neither machine can do the job alone. The controller has no sense of time; the counter has no sense of what time is *for*.

Step 2: Establish How They Communicate

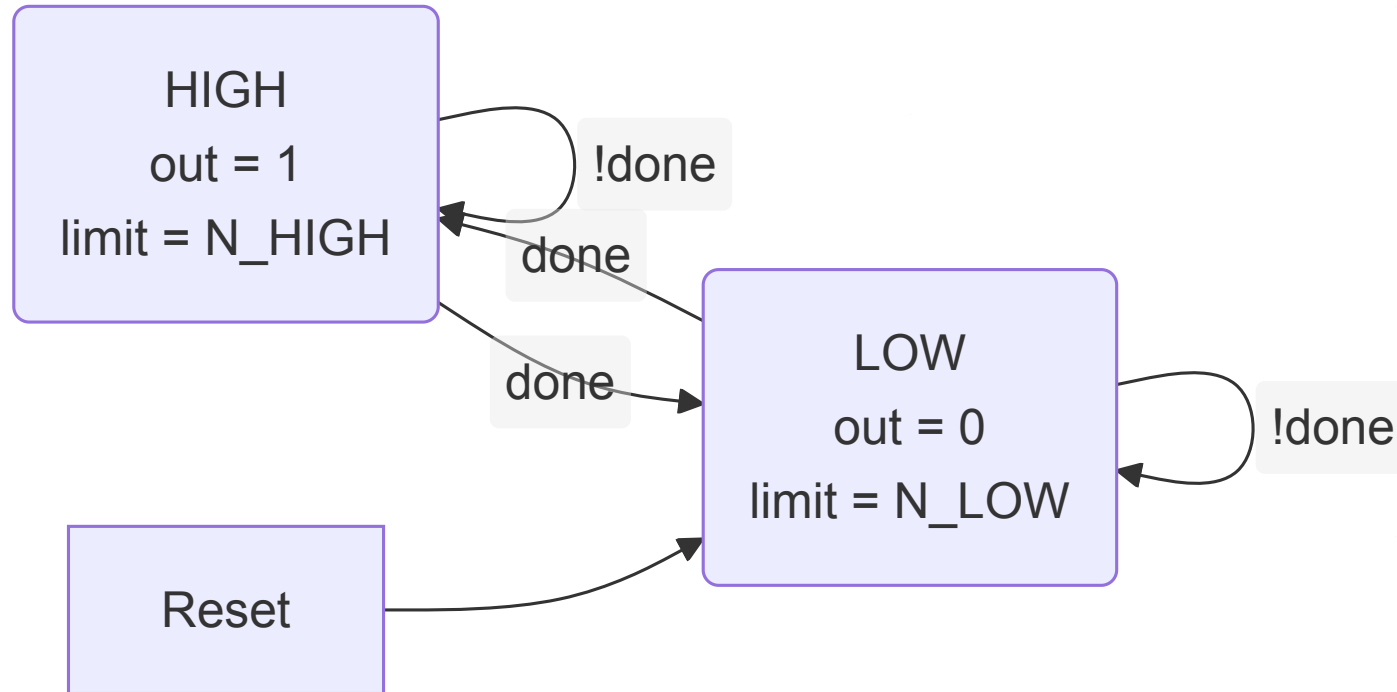
Define the signals that go between the two state machines.

- **Down** — **limit**: how far to count. It depends on the current state, and that is exactly what buys us an adjustable duty cycle.
- **Up** — **done**: the counter has reached **limit**. The controller uses it as its only transition condition.

```
1 assign limit = (state == HIGH) ? N_HIGH : N_LOW;  
2 assign done  = (count == limit - 1);
```

A state-dependent limit is the one new idea here. The debouncer we build next counts to the same place every time; this counter counts to a different place depending on which state is asking.

Step 3: The State Transition Diagram



Two bubbles. All the timing lives in **done**.

Steps 4-7: Declarations and the Counter

```
1  module pwm #(parameter N_HIGH = 12_000, N_LOW = 36_000)
2      (input logic clk, reset, output logic out);
3
4      typedef enum logic {HIGH, LOW} statetype;
5      statetype state, nextstate;
6      logic [15:0] count, limit;
7      logic done;
8
9      // ---- the conversation between the two machines ----
10     assign limit = (state == HIGH) ? N_HIGH : N_LOW;
11     assign done  = (count == limit - 1);
12
13     // ---- machine 2: the counter (non-canonical) ----
14     always_ff @(posedge clk, posedge reset)
15         if (reset)      count <= 0;
16         else if (done) count <= 0;      // the transition clears it
17         else            count <= count + 1;
```

Steps 4-7: The Controller

```
1    // ---- machine 1: the controller (canonical) ----
2    always_ff @(posedge clk, posedge reset)
3        if (reset) state <= LOW;
4        else      state <= nextstate;
5
6    always_comb
7        case (state)
8            HIGH:    nextstate = done ? LOW  : HIGH;
9            LOW:     nextstate = done ? HIGH : LOW;
10           default: nextstate = LOW;
11        endcase
12
13    assign out = (state == HIGH);
14 endmodule
```

Choosing the Parameters

Target: 1 kHz at 25% duty, from 48 MHz.

- Cycles per period: $48,000,000 / 1,000 = 48,000$
- $N_H: 0.25 \times 48,000 = 12,000$
- $N_L: 48,000 - 12,000 = 36,000$
- Counter width: 16 bits, since it has to reach 36,000 and $2^{16} = 65,536$

Q: Why `count == limit - 1` and not `count == limit`?

A: Counting 0, 1, ..., limit-1 is `limit` cycles. Compare against `limit` and every state lasts one cycle too long.

Where You Will Use This

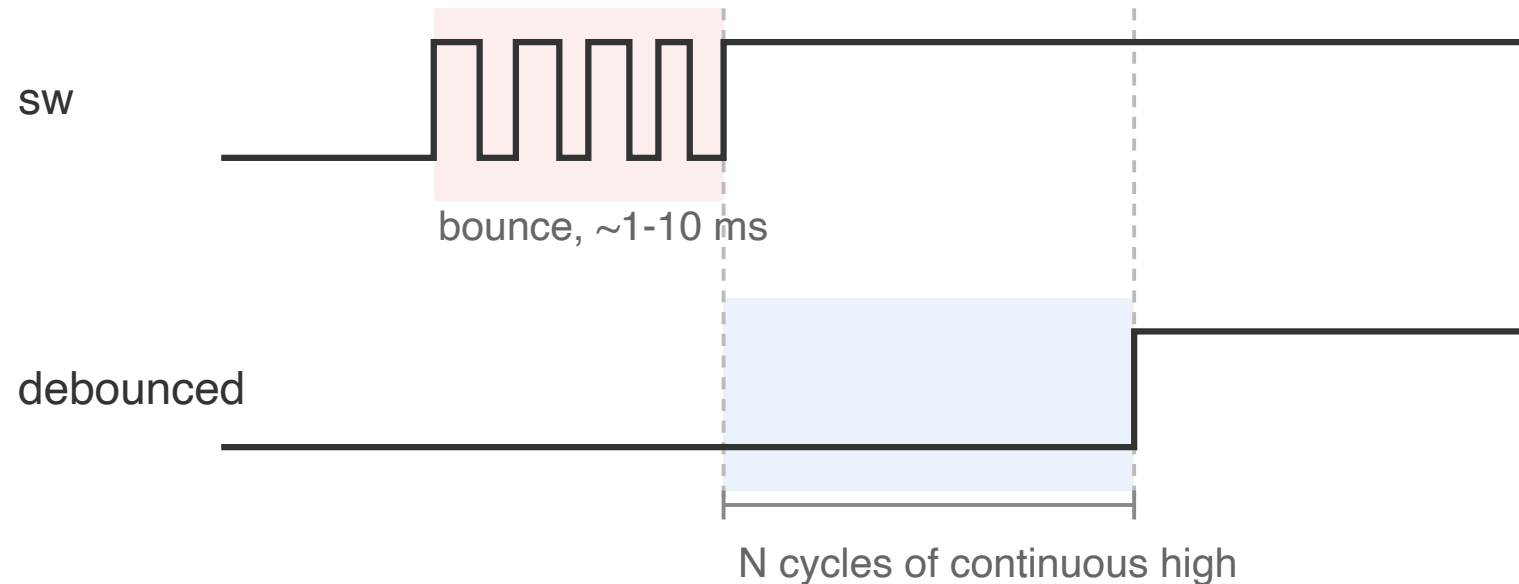
- Dimming an LED
- Motor speed and servo position
- Generating a tone on a speaker
- Lab 5, and very likely your project

Change one parameter and the same circuit does a different job. The controller never changes — only the numbers the counter is told to reach.

Notice you now have all three types in one small module: a canonical controller, a non-canonical counter, and a two-wire conversation between them.

FSM Build 1: Switch Debouncer

Why Your Switch Lies To You



- Mechanical contacts do not close cleanly. They chatter for **1–10 ms** before settling.
- On Lab 3's 150 Hz scan clock you mostly get away with it. At the full 48 MHz, 10 ms of bounce is **about 480,000 clock edges** — one press, registered hundreds of times.

Build 1: The Specification

Task: `debounced_sw` goes high only after `sw` has been continuously high for `N` clock cycles. It returns low as soon as `sw` goes low.

A bounce is, by definition, a high that does not *stay* high. So the machine has to be able to *give up and go back* partway through.

We will do this one together, on the board. Build 2 you will do in your groups.

Step 1: List the Specifications

Inputs

- `sw` — the raw switch, synchronized first. Bounce is not metastability, and a synchronizer does not fix it.
- `clk, reset`

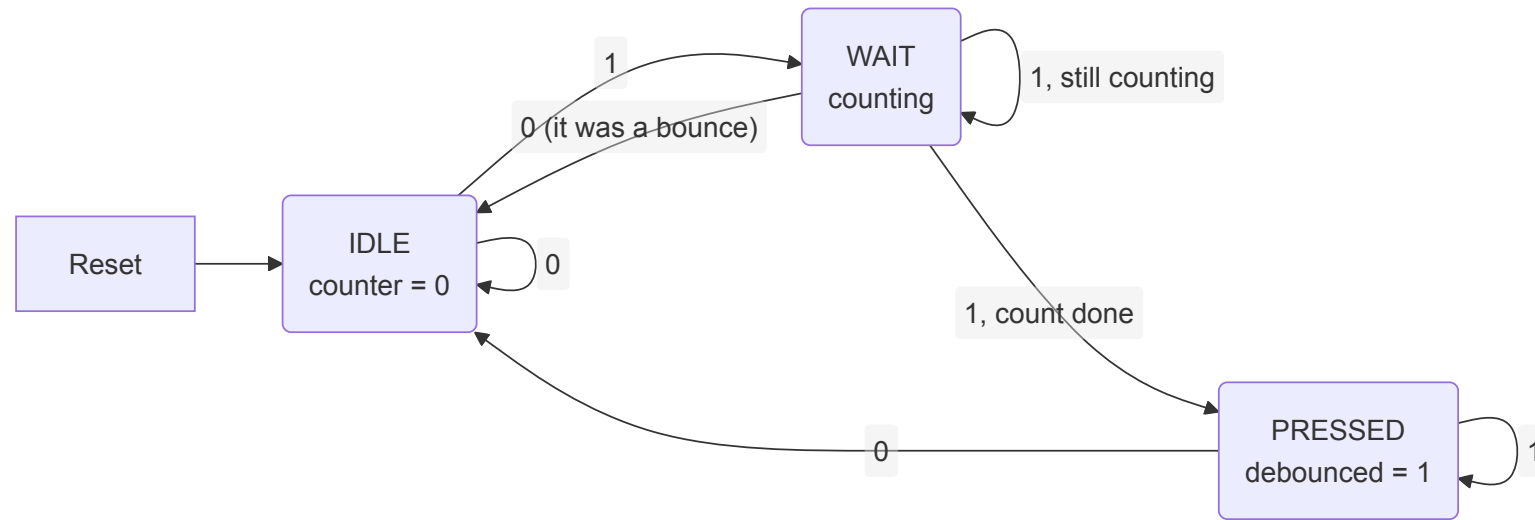
Outputs

- `debounced_sw`

Internal

- `counter` — our first design where the FSM has to *drive* a datapath element, not just read inputs.

Step 2: Sketch the State Transition Diagram



Edge labels are the value of **sw**.

Step 3: A State That Takes Time

Every state we have drawn so far lasted one clock cycle, or sat waiting on an input. **WAIT** is different: it waits on a **counter**, and the coupling runs both directions.

- The FSM **resets** the counter (in **IDLE**)
- The counter **decides** a transition (**WAIT** → **PRESSED**)

Q: Why reset the counter in **IDLE** rather than on the way into **WAIT**?

A: Here they amount to the same thing, and **IDLE** is cheaper to decode. What matters is that it is reset *somewhere*: a free-running counter would let a bounce through whenever it happened to be near the top of its range.

Steps 4-7: State Register and Counter

```
1  typedef enum logic [1:0] {IDLE, WAIT, PRESSED} statetype;
2  statetype state, nextstate;
3
4  logic [19:0] counter;
5
6  always_ff @(posedge clk, posedge reset)
7      if (reset) state <= IDLE;
8      else      state <= nextstate;
9
10 // The FSM owns the counter: cleared in IDLE, running everywhere else.
11 always_ff @(posedge clk)
12     if (state == IDLE) counter <= 0;
13     else               counter <= counter + 1;
```

Steps 4-7: Next State and Output

```
1  always_comb
2      case (state)
3          IDLE:    nextstate = sw ? WAIT : IDLE;
4          WAIT:    if (!sw)          nextstate = IDLE;      // a bounce
5                   else if (counter[19]) nextstate = PRESSED;
6                   else              nextstate = WAIT;
7          PRESSED: nextstate = sw ? PRESSED : IDLE;
8          default: nextstate = IDLE;
9      endcase
10
11 assign debounced_sw = (state == PRESSED);
```

How Big Should the Counter Be?

At 48 MHz, rejecting 10 ms of bounce needs $0.010 \times 48,000,000 = 480,000$ cycles.

Round up to the next power of two: $2^{19} = 524,288$, about 10.9 ms — so a 20-bit counter, and we watch bit 19.

Q: Why test `counter[19]` instead of `counter == 20'd480000`?

A: Tapping a bit is **free** — it is a wire. A 20-bit equality comparator is real logic, several LEs of it. The price is that you only get thresholds that are powers of two, and for debouncing nobody cares whether the wait is exactly 10 ms or 10.9.

This is the whole game in hardware design: a constraint that looks arbitrary (why a power of two?) is usually somebody trading precision they did not need for logic they did.

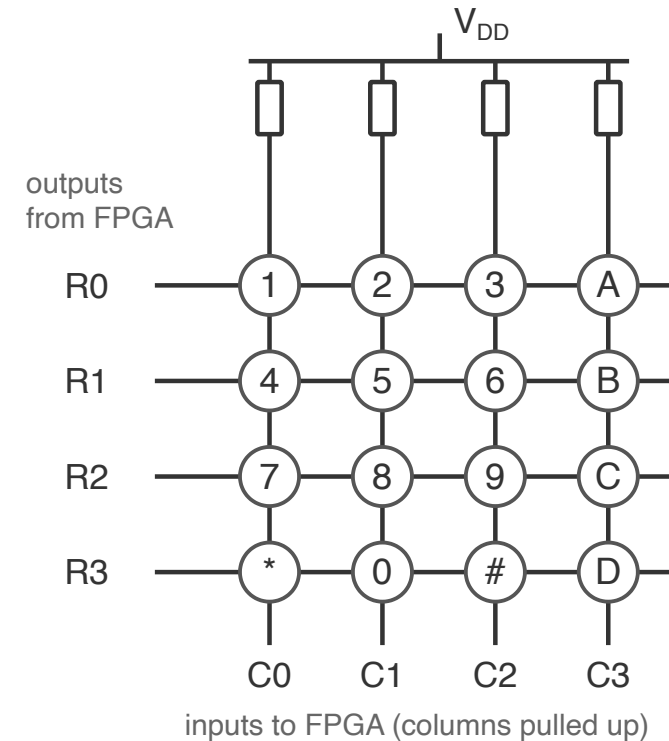
FSM Build 2: Keypad Scanner

Lab 3 Preview: Reading a 4x4 Keypad

Lab 3 asks you to read a 16-key keypad and display the last two keys pressed on your seven-segment display.

- 16 keys, but only **8** pins: 4 rows + 4 columns
- Drive **one row low** at a time
- Each column is **pulled up**, so a column reads 1 unless a key in the driven row is pressed
- Pressing the key at (row r , col c) shorts row r to column c

`cols == 4'b1111` therefore means **no key is pressed in the row you are currently driving** — it says nothing about the other three rows.



Why Scan at All?

Q: Why not just give every key its own pin?

A: 16 keys would need 16 pins plus a ground. Scanning trades a few gates and some time for pins, and pins are the scarce resource – we only have 39 of them.

Q: We scan the rows at about 150 Hz. Why not run the scanner at the full 48 MHz oscillator speed?

A: Nothing breaks, but there is no point: fingers are slow. The slow clock also stretches each scan step to ~6.7 ms, comparable to the bounce time, which takes the edge off the problem — but it does not remove it. That is why Lab 3 wants the debouncer from Build 1 as well.

Your Task

Design the FSM that drives the scan.

Given: `clk`, `reset`, and `cols[3:0]` (already synchronized — you wrote that synchronizer last lecture).

Produce: `rows[3:0]` rotating one-hot-low, and `d0/d1` holding the last two keys pressed.

Requirements

1. One key press registers **exactly once**, no matter how long it is held.
2. While a key is held, ignore any other key.
3. All outputs are registered — no glitches on `rows`.

Work in groups of 3. Use the six-step process from Lecture 04.

Step 1: List the Specifications

Inputs

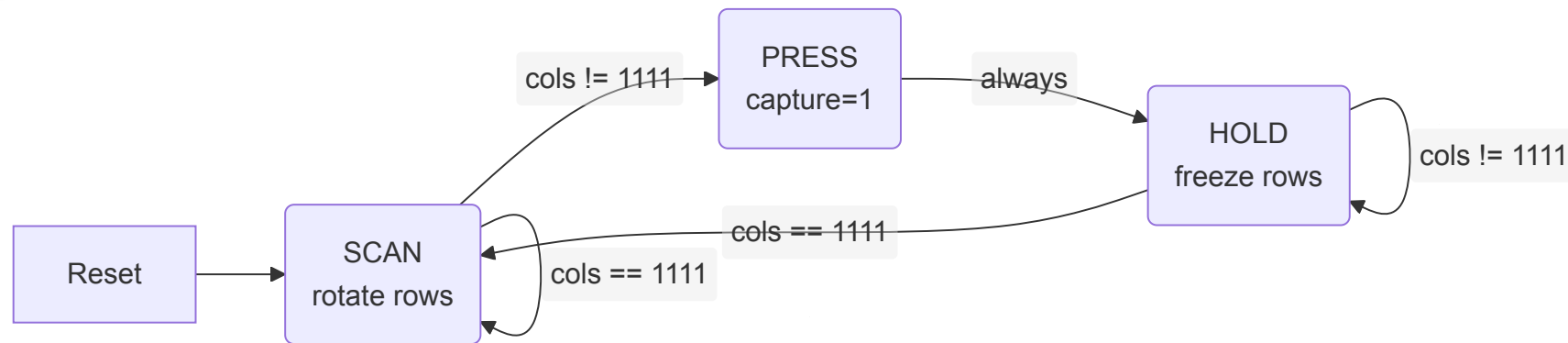
- `cols[3:0]`, synchronized. Really we only need one bit of it for the state logic: `any_key = ~&cols`
- `clk`, `reset`

Outputs

- `rows[3:0]` — rotating one-hot-low scan pattern
- `d0[3:0]`, `d1[3:0]` — the last two keys
- Internally: a `capture` signal that says “latch the key now”

Step 2: Sketch the State Transition Diagram

Hint: ask yourself what the machine has to *remember*. It is not the key — that lives in a register. It is **where you are in the press: still looking, just found one, or waiting for the finger to come off.**



The Same Three States, Again

	Level-to-pulse (L04)	Debouncer (Build 1)	Keypad scanner (Build 2)
idle	S0 — input low	IDLE — switch up, counter cleared	SCAN — no key, rotate rows
transitional	S1 — emit pulse	WAIT — count out the bounce	PRESS — capture the key
held	S2 — wait for release	PRESSED — hold output high	HOLD — wait for release

The same three states every time. What actually changes is (a) how long the middle state lasts — one cycle, or **N** — and (b) what the machine does to its datapath while it sits there. Recognize the skeleton and the six steps go fast; the real design work is in (a) and (b).

Step 3: State Transition Table

Current State	any_key	Next State	rows	capture
SCAN	0	SCAN	rotate	0
SCAN	1	PRESS	freeze	0
PRESS	0	HOLD	freeze	1
PRESS	1	HOLD	freeze	1
HOLD	0	SCAN	freeze	0
HOLD	1	HOLD	freeze	0

Note PRESS goes to HOLD either way. It is a one-cycle state, exactly like S1 last lecture.

Step 4-7: Encode and Implement

```
1  typedef enum logic [2:0] {SCAN = 3'b001, PRESS = 3'b010,  
2                               HOLD = 3'b100} statetype;  
3  statetype state, nextstate;  
4  logic any_key;  
5  
6  assign any_key = ~&cols;      // low-asserted: any column pulled down  
7  
8  always_ff @(posedge clk, posedge reset)  
9      if (reset) state <= SCAN;  
10     else      state <= nextstate;  
11  
12  always_comb  
13      case (state)  
14          SCAN:    nextstate = any_key ? PRESS : SCAN;  
15          PRESS:   nextstate = HOLD;  
16          HOLD:    nextstate = any_key ? HOLD : SCAN;  
17          default: nextstate = SCAN;  
18      endcase
```

Step 4-7: Encode and Implement

The datapath hangs off the state, and all of it is registered:

```
1  always_ff @(posedge clk, posedge reset)
2      if (reset) begin
3          rows <= 4'b0111;
4          d0    <= 4'h0;
5          d1    <= 4'h0;
6      end else begin
7          // Only SCAN moves the row pattern. Everything else freezes it.
8          if (state == SCAN && !any_key) rows <= {rows[0], rows[3:1]};
9
10         if (state == PRESS) begin
11             d1 <= d0;
12             d0 <= key;      // combinational decode of {rows, cols}
13         end
14     end
```

Testing This Is Harder Than It Looks

In hardware `cols` is not an independent input — it **responds** to `rows`. A testbench that wiggles `cols` on a clock schedule will happily produce key combinations that no finger can create.

Model the keypad instead. One key held at (row R , col C) means:

```
1 // Key '5' sits at row 1, column 1. Holding it ties column 1 to row 1;
2 // every other column stays pulled up.
3 logic key5_down;
4
5 assign cols[1] = key5_down ? rows[1] : 1'b1;
6 assign cols[0] = 1'b1;
7 assign cols[2] = 1'b1;
8 assign cols[3] = 1'b1;
```

Now press and release `key5_down` on a slow schedule and let the DUT scan at its own pace. **Hold it for hundreds of cycles and confirm `d0` changes exactly once.**

How Many Logic Cells Did That Cost?

One logic cell on our FPGA is a 4-input LUT plus a flip-flop, so a registered function of up to 4 inputs is 1 LE.

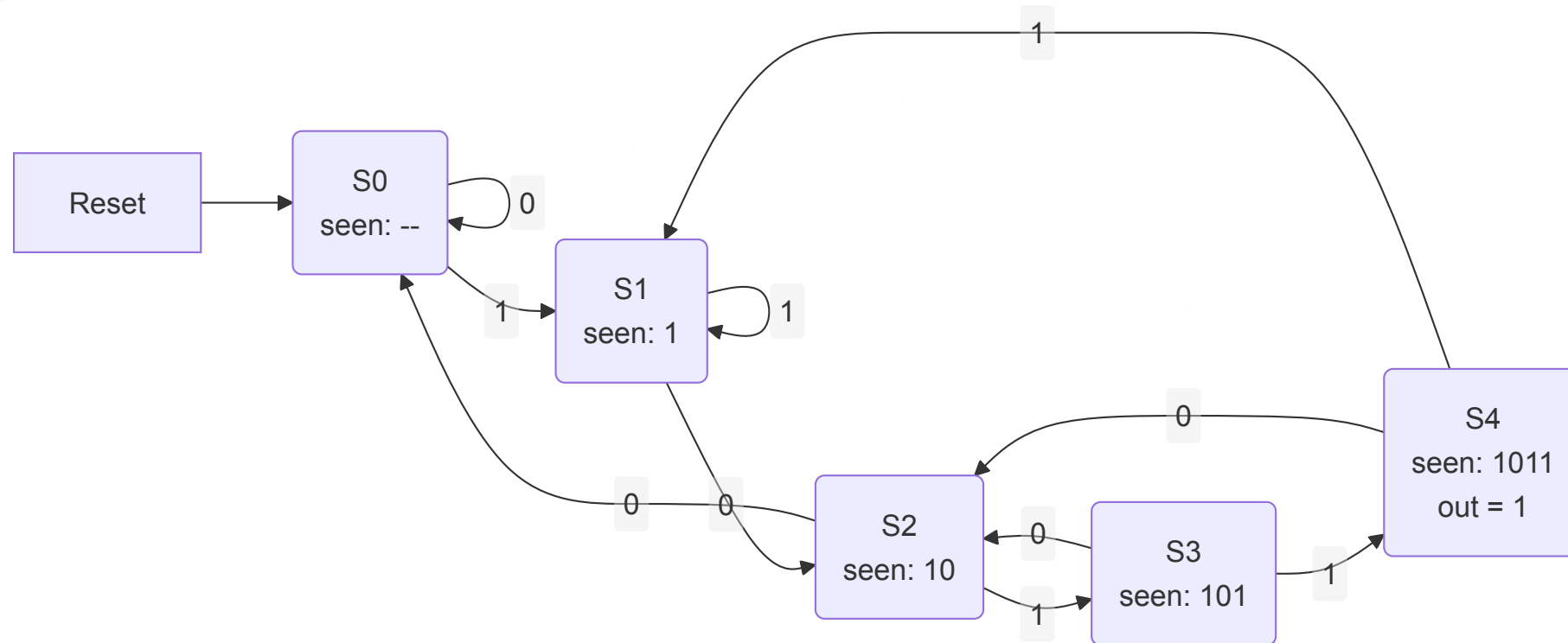
- State register, one-hot, 3 states: 3 LEs. Each next-state bit is a function of 3 state bits + any_key = 4 inputs, so 1 LUT each — and the flop rides along in the same LC for free.
- rows rotating register: 4 LEs
- d0 and d1: 8 LEs
- key decoder, 4 outputs from {rows, cols} (8 inputs): An arbitrary 8-input function takes 31 LUTs (16 LUTs for the 4-input cofactors, then 15 more to mux them down), so 124 LEs as an upper bound. The real decoder is nowhere near arbitrary — only 16 of the 256 input combinations are legal — so the synthesizer does far better.

Build it, then open the Radiant synthesis report and compare. The gap between your hand estimate and the tool's number is the interesting part.

When It Is Not This Skeleton

A Machine That Does Not Fit the Pattern

Detect the bit pattern **1011** in a serial stream, including overlapping matches.



No amount of squinting turns this into idle / transition / held. The machine has to remember **how much of the pattern it has already seen** — that is genuinely new state, not a rephrasing of “is the input still asserted?”

Wrap up: Metastability

- Metastability cannot be eliminated, only made improbable. MTBF $= T_c e^{(T_c - t_{setup})/\tau} / (NT_0)$ is exponential in how long you wait and linear in everything else, which is why the fix is always one more clock cycle.
- A synchronizer fixes metastability and nothing else. It will not debounce, and it will not keep a multi-bit value coherent.

Wrap up: State Machines

- Canonical, non-canonical, communicating: same hardware underneath, three different ways to describe it. Pick by whether the next state is easier to draw or to compute.
- Control plus datapath is the idea that makes big designs small. A 2-state controller steering a 36,000-state counter is two easy problems instead of one impossible one.
- Three of the four machines we have now looked at share one skeleton: idle, transitional, held. Most FSMs you meet are a machine you have seen before plus a datapath.
- Not all of them, though. When a machine has to remember *how much* of something it has seen, that is real state and no skeleton saves you.
- An FSM that drives a datapath element owns that element: something has to reset the counter, and it should be the state machine.
- When an input responds to your own outputs, your testbench has to model that feedback or it will verify a system that cannot exist.

Announcements/Reminders

- **Lab 3 (keypad scanner)** is posted. *Both* machines we built today are modules in it — start from your state diagrams, not from a blank `.sv` file.
- Reuse your Lab 2 `seven_seg` module and your Lab 4 synchronizer. `cols` is asynchronous and must be synchronized before it reaches the FSM.
- Simulate before you synthesize. Model the keypad in your testbench as shown today.
- Lab 2 checkoffs wrap up this week.